

Floyd 算法

是用来求任意两个结点之间的最短路的。

复杂度比较高，但是常数小，容易实现（只有三个 `for`）。

适用于任何图，不管有向无向，边权正负，但是最短路必须存在。（不能有个负环）

实现 ¶

我们定义一个数组 $f[k][x][y]$ ，表示只允许经过结点 1 到 k （也就是说，在子图 $V' = 1, 2, \dots, k$ 中的路径，注意， x 与 y 不一定在这个子图中），结点 x 到结点 y 的最短路长度。



很显然， $f[n][x][y]$ 就是结点 x 到结点 y 的最短路长度（因为 $V' = 1, 2, \dots, n$ 即为 V 本身，其表示的最短路径就是所求路径）。

接下来考虑如何求出 f 数组的值。

$f[0][x][y]$ ： x 与 y 的边权，或者 0，或者 $+\infty$ （ $f[0][x][y]$ 什么时候应该是 $+\infty$ ？当 x 与 y 间有直接相连的边的时候，为它们的边权；当 $x = y$ 的时候为零，因为到本身的距离为零；当 x 与 y 没有直接相连的边的时候，为 $+\infty$ ）。

$f[k][x][y] = \min(f[k-1][x][y], f[k-1][x][k] + f[k-1][k][y])$ ($f[k-1][x][y]$ ，为不经过 k 点的最短路径，而 $f[k-1][x][k] + f[k-1][k][y]$ ，为经过了 k 点的最短路)。

上面两行都显然是对的，所以说这个做法空间是 $O(N^3)$ ，我们需要依次增加问题规模 (k 从 1 到 n)，判断任意两点在当前问题规模下的最短路。

C++ **Python**

```
1  for (k = 1; k <= n; k++) {
2      for (x = 1; x <= n; x++) {
3          for (y = 1; y <= n; y++) {
4              f[k][x][y] = min(f[k - 1][x][y], f[k - 1][x][k] + f[k - 1][k][y]);
5          }
6      }
7  }
```

因为第一维对结果无影响，我们可以发现数组的第一维是可以省略的，于是可以直接改成 `f[x][y] = min(f[x][y], f[x][k]+f[k][y])`。

证明第一维对结果无影响

对于给定的 `k`，当更新 `f[k][x][y]` 时，涉及的元素总是来自 `f[k-1]` 数组的第 `k` 行和第 `k` 列。然后我们可以发现，对于给定的 `k`，当更新 `f[k][k][y]` 或 `f[k][x][k]`，总是不会发生数值更新，因为按照公式 `f[k][k][y] = min(f[k-1][k][y], f[k-1][k][k]+f[k-1][k][y])`，`f[k-1][k][k]` 为 0，因此这个值总是 `f[k-1][k][y]`，对于 `f[k][x][k]` 的证明类似。

因此，如果省略第一维，在给定的 `k` 下，每个元素的更新中使用到的元素都没有在这次迭代中更新，因此第一维的省略并不会影响结果。

C++**Python**

```
1  for (k = 1; k <= n; k++) {  
2      for (x = 1; x <= n; x++) {  
3          for (y = 1; y <= n; y++) {  
4              f[x][y] = min(f[x][y], f[x][k] + f[k][y]);  
5          }  
6      }  
7  }
```



综上时间复杂度是 $O(N^3)$ ，空间复杂度是 $O(N^2)$ 。

Dijkstra 算法

Dijkstra (/ˈdɪkstrə/或/ˈdeɪkstrə/) 算法由荷兰计算机科学家 E. W. Dijkstra 于 1956 年发现，1959 年公开发表。是一种求解 **非负权图** 上单源最短路径的算法。

- 1.首先，指定一个起点（源点 s ），并将它到自身的距离设为0，其他所有点到源点的距离初始化为无穷大。
- 2.然后，从所有未被访问的顶点中，选择一个距离源点最近的顶点 u 。
- 3.将顶点 u 标记为“已访问”。
- 4.遍历 u 的所有邻接点 v ，进行“松弛”操作：如果从源点 s 经过 u 到达 v 的距离 ($\text{dis}[u] + w$) 比当前记录的 s 到 v 的距离 ($\text{dis}[v]$) 更短，那么就更新 $\text{dis}[v]$ 的值。
- 5.重复步骤 2-4，直到所有顶点都被访问过。最终， dis 数组中就存储了源点 s 到所有其他顶点的最短距离。

```
1  struct edge {
2      int v, w;
3  };
4
5  vector<edge> e[MAXN];
6  int dis[MAXN], vis[MAXN];
7
8  void dijkstra(int n, int s) {
9      memset(dis, 0x3f, (n + 1) * sizeof(int));
10     dis[s] = 0;
11     for (int i = 1; i <= n; i++) {
12         int u = 0, mind = 0x3f3f3f3f;
13         for (int j = 1; j <= n; j++)
14             if (!vis[j] && dis[j] < mind) u = j, mind = dis[j];
15         vis[u] = true;
16         for (auto ed : e[u]) {
17             int v = ed.v, w = ed.w;
18             if (dis[v] > dis[u] + w) dis[v] = dis[u] + w;
19         }
20     }
21 }
```

